Practical No. 09

Aim: Implementation of different types of **Joins**.

Software Requirements: MySQL, Oracle 10g.

Theory:

A **Join** in SQL is used to combine rows from two or more tables based on a related column between them.

Types of Joins

1. **INNER JOIN**
    - Returns only the rows with matching values in both tables.
    - Example: Fetch employees who belong to an existing department.
2. **LEFT JOIN (LEFT OUTER JOIN)**
    - Returns all rows from the left table and the matching rows from the right table.
    - If no match exists, NULL values are returned for the right table's columns.
3. **RIGHT JOIN (RIGHT OUTER JOIN)**
    - Returns all rows from the right table and the matching rows from the left table.
    - If no match exists, NULL values are returned for the left table's columns.
4. **FULL OUTER JOIN**
    - Combines the results of both LEFT and RIGHT joins.
    - Returns all rows when there is a match in either left or right table.
    - In MySQL, simulated using UNION.
5. **CROSS JOIN**
    - Returns the Cartesian product of the two tables.
    - If one table has m rows and another has n rows, the result will have $m \times n$ rows.
6. **Multi-Table Joins**
    - We can join more than two tables together (e.g., Employees + Departments + Projects).

Program:

```
CREATE DATABASE joins_demo_large;
USE joins_demo_large;

CREATE TABLE departments (
  dept_id INT PRIMARY KEY,
  dept_name VARCHAR(50),
  location VARCHAR(50)
);

INSERT INTO departments VALUES
(1, 'CSE', 'Block A'),
(2, 'IT', 'Block B'),
(3, 'AI', 'Block C'),
(4, 'DS', 'Block D'),
(5, 'HR', 'Block E');

SELECT * FROM departments;

CREATE TABLE employees (
  emp_id INT PRIMARY KEY,
  name VARCHAR(50),
  dept_id INT,
  salary DECIMAL(10,2),
  FOREIGN KEY (dept_id) REFERENCES departments(dept_id)
);

INSERT INTO employees VALUES
(101, 'Alice', 1, 60000),
(102, 'Bob', 2, 45000),
(103, 'Charlie', 3, 70000),
(104, 'David', 1, 50000),
(105, 'Eva', 4, 65000),
(106, 'Frank', NULL, 40000);

SELECT * FROM employees;

CREATE TABLE projects (
  project_id INT PRIMARY KEY,
  emp_id INT,
  project_name VARCHAR(50),
  FOREIGN KEY (emp_id) REFERENCES employees(emp_id)
);

INSERT INTO projects VALUES
(201, 101, 'Library System'),
(202, 102, 'Payroll System'),
(203, 104, 'Attendance App'),
(204, 105, 'Data Warehouse');
```

```
SELECT * FROM projects;

SELECT e.emp_id, e.name, d.dept_name, e.salary
FROM employees e
INNER JOIN departments d ON e.dept_id = d.dept_id;

SELECT e.emp_id, e.name, d.dept_name, e.salary
FROM employees e
LEFT JOIN departments d ON e.dept_id = d.dept_id;

SELECT e.emp_id, e.name, d.dept_name
FROM employees e
RIGHT JOIN departments d ON e.dept_id = d.dept_id;

SELECT e.emp_id, e.name, d.dept_name
FROM employees e
LEFT JOIN departments d ON e.dept_id = d.dept_id
UNION
SELECT e.emp_id, e.name, d.dept_name
FROM employees e
RIGHT JOIN departments d ON e.dept_id = d.dept_id;

SELECT e.name, p.project_name, d.dept_name
FROM employees e
INNER JOIN projects p ON e.emp_id = p.emp_id
INNER JOIN departments d ON e.dept_id = d.dept_id;

SELECT e.name, d.dept_name
FROM employees e
CROSS JOIN departments d;
```

Output:

```
mysql>
mysql> -- 1. INNER JOIN (Employees with Departments)
mysql> SELECT e.emp_id, e.name, d.dept_name, e.salary
    -> FROM employees e
    -> INNER JOIN departments d ON e.dept_id = d.dept_id;
+--------+---------+-----------+-----------+
| emp_id | name    | dept_name | salary    |
+--------+---------+-----------+-----------+
|    101 | Alice   | CSE       | 60000.00 |
|    104 | David   | CSE       | 50000.00 |
|    102 | Bob     | IT        | 45000.00 |
|    103 | Charlie | AI        | 70000.00 |
|    105 | Eva     | DS        | 65000.00 |
+--------+---------+-----------+-----------+
5 rows in set (0.00 sec)

mysql>
mysql> -- 2. LEFT JOIN (All Employees, with Dept if available)
mysql> SELECT e.emp_id, e.name, d.dept_name, e.salary
    -> FROM employees e
    -> LEFT JOIN departments d ON e.dept_id = d.dept_id;
+--------+---------+-----------+-----------+
| emp_id | name    | dept_name | salary    |
+--------+---------+-----------+-----------+
|    101 | Alice   | CSE       | 60000.00 |
|    102 | Bob     | IT        | 45000.00 |
|    103 | Charlie | AI        | 70000.00 |
|    104 | David   | CSE       | 50000.00 |
|    105 | Eva     | DS        | 65000.00 |
|    106 | Frank   | NULL      | 40000.00 |
+--------+---------+-----------+-----------+
6 rows in set (0.00 sec)

mysql>
mysql> -- 3. RIGHT JOIN (All Departments, with Employees if available)
mysql> SELECT e.emp_id, e.name, d.dept_name
    -> FROM employees e
    -> RIGHT JOIN departments d ON e.dept_id = d.dept_id;
+--------+---------+-----------+
| emp_id | name    | dept_name |
+--------+---------+-----------+
|    101 | Alice   | CSE       |
|    104 | David   | CSE       |
|    102 | Bob     | IT        |
|    103 | Charlie | AI        |
|    105 | Eva     | DS        |
|   NULL | NULL    | HR        |
+--------+---------+-----------+
6 rows in set (0.00 sec)

mysql>
mysql> -- 4. FULL OUTER JOIN (Simulated with UNION in MySQL)
mysql> SELECT e.emp_id, e.name, d.dept_name
    -> FROM employees e
    -> LEFT JOIN departments d ON e.dept_id = d.dept_id
    -> UNION
    -> SELECT e.emp_id, e.name, d.dept_name
    -> FROM employees e
    -> RIGHT JOIN departments d ON e.dept_id = d.dept_id;
+--------+---------+-----------+
| emp_id | name    | dept_name |
+--------+---------+-----------+
|    101 | Alice   | CSE       |
|    102 | Bob     | IT        |
|    103 | Charlie | AI        |
|    104 | David   | CSE       |
|    105 | Eva     | DS        |
|    106 | Frank   | NULL      |
|   NULL | NULL    | HR        |
+--------+---------+-----------+
7 rows in set (0.00 sec)

mysql>
mysql> -- 5. MULTI-TABLE JOIN (Employees with Projects and Departments)
mysql> SELECT e.name, p.project_name, d.dept_name
    -> FROM employees e
    -> INNER JOIN projects p ON e.emp_id = p.emp_id
    -> INNER JOIN departments d ON e.dept_id = d.dept_id;
+-------+----------------+-----------+
| name  | project_name   | dept_name |
+-------+----------------+-----------+
| Alice | Library System | CSE       |
| Bob   | Payroll System | IT        |
| David | Attendance App | CSE       |
| Eva   | Data Warehouse | DS        |
+-------+----------------+-----------+
4 rows in set (0.00 sec)
```

```
mysql> -- Create Database
mysql> CREATE DATABASE joins_demo_large;
Query OK, 1 row affected (0.02 sec)

mysql> USE joins_demo_large;
Database changed
mysql>
mysql> -- Create Departments Table
mysql> CREATE TABLE departments (
    ->    dept_id INT PRIMARY KEY,
    ->    dept_name VARCHAR(50),
    ->    location VARCHAR(50)
    -> );
Query OK, 0 rows affected (0.05 sec)

mysql>
mysql> -- Insert Departments
mysql> INSERT INTO departments VALUES
    -> (1, 'CSE', 'Block A'),
    -> (2, 'IT', 'Block B'),
    -> (3, 'AI', 'Block C'),
    -> (4, 'DS', 'Block D'),
    -> (5, 'HR', 'Block E');
Query OK, 5 rows affected (0.01 sec)
Records: 5  Duplicates: 0  Warnings: 0

mysql>
mysql> -- Show Departments Table
mysql> SELECT * FROM departments;
+---------+-----------+----------+
| dept_id | dept_name | location |
+---------+-----------+----------+
|       1 | CSE       | Block A  |
|       2 | IT        | Block B  |
|       3 | AI        | Block C  |
|       4 | DS        | Block D  |
|       5 | HR        | Block E  |
+---------+-----------+----------+
5 rows in set (0.00 sec)

mysql>
mysql> -- Create Employees Table
mysql> CREATE TABLE employees (
    ->    emp_id INT PRIMARY KEY,
    ->    name VARCHAR(50),
    ->    dept_id INT,
    ->    salary DECIMAL(10,2),
    ->    FOREIGN KEY (dept_id) REFERENCES departments(dept_id)
    -> );
Query OK, 0 rows affected (0.06 sec)

mysql>
mysql> -- Insert Employees
mysql> INSERT INTO employees VALUES
    -> (101, 'Alice', 1, 60000),
    -> (102, 'Bob', 2, 45000),
    -> (103, 'Charlie', 3, 70000),
    -> (104, 'David', 1, 50000),
    -> (105, 'Eva', 4, 65000),
    -> (106, 'Frank', NULL, 40000); -- Employee without department
Query OK, 6 rows affected (0.01 sec)
Records: 6  Duplicates: 0  Warnings: 0

mysql>
mysql> -- Show Employees Table
mysql> SELECT * FROM employees;
+--------+---------+---------+----------+
| emp_id | name    | dept_id | salary   |
+--------+---------+---------+----------+
|    101 | Alice   |       1 | 60000.00 |
|    102 | Bob     |       2 | 45000.00 |
|    103 | Charlie |       3 | 70000.00 |
|    104 | David   |       1 | 50000.00 |
|    105 | Eva     |       4 | 65000.00 |
|    106 | Frank   |    NULL | 40000.00 |
+--------+---------+---------+----------+
6 rows in set (0.00 sec)

mysql>
mysql> -- Create Projects Table
mysql> CREATE TABLE projects (
    ->    project_id INT PRIMARY KEY,
    ->    emp_id INT,
    ->    project_name VARCHAR(50),
    ->    FOREIGN KEY (emp_id) REFERENCES employees(emp_id)
    -> );
Query OK, 0 rows affected (0.07 sec)

mysql>
mysql> -- Insert Projects (only valid emp_id used)
mysql> INSERT INTO projects VALUES
    -> (201, 101, 'Library System'),
    -> (202, 102, 'Payroll System'),
    -> (203, 104, 'Attendance App'),
    -> (204, 105, 'Data Warehouse'); -- valid employee Eva
Query OK, 4 rows affected (0.01 sec)
Records: 4  Duplicates: 0  Warnings: 0

mysql>
mysql> -- Show Projects Table
mysql> SELECT * FROM projects;
+------------+--------+----------------+
| project_id | emp_id | project_name   |
+------------+--------+----------------+
|        201 |    101 | Library System |
|        202 |    102 | Payroll System |
|        203 |    104 | Attendance App |
|        204 |    105 | Data Warehouse |
+------------+--------+----------------+
4 rows in set (0.00 sec)
```

```
mysql>
mysql> -- 6. CROSS JOIN (All combinations of Employees × Departments)
mysql> SELECT e.name, d.dept_name
    -> FROM employees e
    -> CROSS JOIN departments d;
+---------+-----------+
| name    | dept_name |
+---------+-----------+
| Alice   | HR        |
| Alice   | DS        |
| Alice   | AI        |
| Alice   | IT        |
| Alice   | CSE       |
| Bob     | HR        |
| Bob     | DS        |
| Bob     | AI        |
| Bob     | IT        |
| Bob     | CSE       |
| Charlie | HR        |
| Charlie | DS        |
| Charlie | AI        |
| Charlie | IT        |
| Charlie | CSE       |
| David   | HR        |
| David   | DS        |
| David   | AI        |
| David   | IT        |
| David   | CSE       |
| Eva     | HR        |
| Eva     | DS        |
| Eva     | AI        |
| Eva     | IT        |
| Eva     | CSE       |
| Frank   | HR        |
| Frank   | DS        |
| Frank   | AI        |
| Frank   | IT        |
| Frank   | CSE       |
+---------+-----------+
30 rows in set (0.00 sec)

mysql>
```

Conclusion:

Hence, we have successfully studied and implemented different types of **Joins**.